



CSE 3313: Computer Architecture

Sidratul Tanzila Tasmi

Lecturer, Department of Computer Science and Engineering
United International University

Processors from the Point of View of Terrible Programmer



Source Code

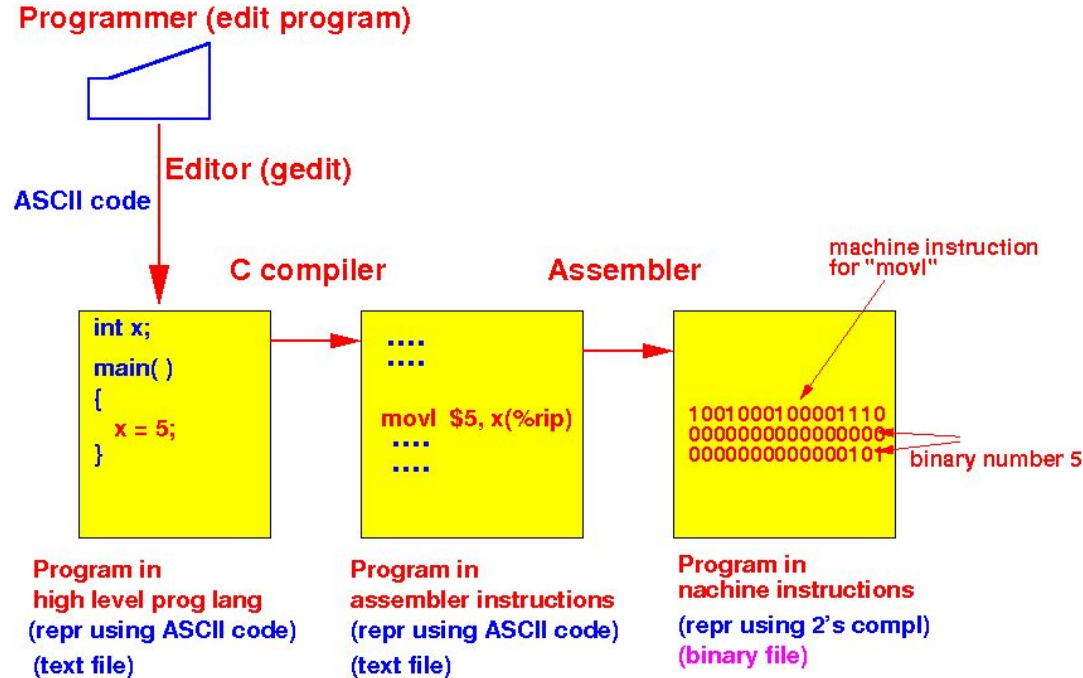


Some Magic
Occurs!



Machine Code

Processors from the Point of View of a good Programmer



Why is a High Level Language converted to assembly language first?



- Compilers generate assembly code first to allow further optimizations before final machine code conversion.
- An assembler's job is to handle architecture-specific encoding of those instructions into machine code.
- **Early compilers (e.g., for C) were designed to produce assembly output because assemblers already existed for converting it to machine code.**

Assembler



- Assembler is a program for converting instructions written in low-level assembly code into relocatable machine code and generating along information for the loader.
- It is necessary to convert user-written programs into machinery code.
- This is called a translation of a high-level language to a low-level that is machinery language

A Simple Assembly Language Program



```
.model small
.stack 100h
.data
    msg db 'Hello, World!$', 0
.code
main proc
    mov ax, @data        ; Initialize data segment
    mov ds, ax
    lea dx, msg          ; Load address of the message
    mov ah, 9            ; Function to display string
    int 21h              ; Call DOS interrupt
    mov ax, 4c00h        ; Exit program
    int 21h
main endp
end main
```

Instruction Set



- The vocabulary of commands understood by a given architecture.
- The set of instructions / primitive operations that a CPU may execute is a major component of ISA (IS Architecture)
- Assembly language is a textual version of these instructions

Instruction Set Architecture



- ISA defines the interface which hardware presents to software
- A compiler translates high-level source code (e.g., C++, Go) to the ISA for a target processor. The processor directly executes ISA instructions
- Example ISAs:
 - MIPS (mostly the focus of 3313: Computer Architecture)
 - ARM (popular on mobile devices)
 - **x86 (popular on desktops and laptops; known to cause sadness among programmers and hardware developers)**

RISC vs CISC Processors



Comparison between RISC and CISC Architectures

S.N.	RISC	CISC
1	Simple instructions taking one cycle	Complex instructions taking multiple cycles
2	Only load and store memory references	Any instructions may reference memory
3	Heavily pipelined	Not/less pipelined
4	Multiple register sets	Single register set
5	Complexity is in compiler	Complexity is in micro-programming
6	Instructions executed by hardware	Instructions interpreted by micro-programming
7	Fixed format instructions	Variable format instructions
8	Few instructions and modes	Large instructions and modes

Microprocessor without Interlocked Pipelined Stages (MIPS)



- MIPS is simple, elegant and similar to other architectures developed since the 1980's
- MIPS widely used in embedded apps
- Almost 100 million MIPS processors manufactured in 2002
- Used by NEC, Nintendo, Cisco, Silicon Graphics, Sony, ...

MIPS Architecture Features



- 32 registers in MIPS, Each MIPS register is 32 bits wide.
Groups of 32 bits called a word in MIPS
- Registers are numbered from 0 to 31, Number references: \$0, \$1, ... \$30, \$31
- Each register can be referred to by number or name. By convention, each register also has a name to make it easier to code \$t0 -\$t7 for temporary variables (\$8-\$15)
- \$ra for return address

MIPS Assembly Language



All instructions have 3 operands

One destination, two source operands

Operand order is fixed (destination first)

Example: C code: $a = b + c$

MIPS code: `add a, b, c`

C code: $a = b + c + d$;

MIPS code: `add a, b, c`

`add a, a, d`



Thank you!